

Creator Payout System Design

Status: Proposed **Created:** 2026-02-28

Overview

This document describes the design for the creator payout system — the mechanism by which LYKE transfers earned commissions and sponsored fees to content creators. It covers the full lifecycle from an unconfirmed affiliate earning to a settled bank transfer, and defines all data model changes, service interfaces, background jobs, external integrations, and new API endpoints required.

Current State

The existing implementation records earnings but does not move money:

Concern	Current State
Click tracking	☑ Complete — <code>ClickEvent</code> created on every outbound product link click
Conversion ingestion	☑ Complete — retailer webhook creates a <code>CreatorEarning</code> with <code>Status = Pending</code>
Attribution window	⚠ Configured (30 days default) but not enforced — earnings never leave <code>Pending</code>
Earnings confirmation	✗ Missing — no job promotes <code>Pending</code> → <code>Confirmed</code>
Payout disbursement	✗ Missing — <code>Creator.PayoutDetails</code> stored but never read; no payment processor
Payout account management	✗ Missing — no onboarding flow to collect banking info
Sponsored earnings	✗ Missing — <code>SponsoredPlacement</code> entity exists but never generates <code>CreatorEarning</code> records

Payment Processor: Stripe Connect

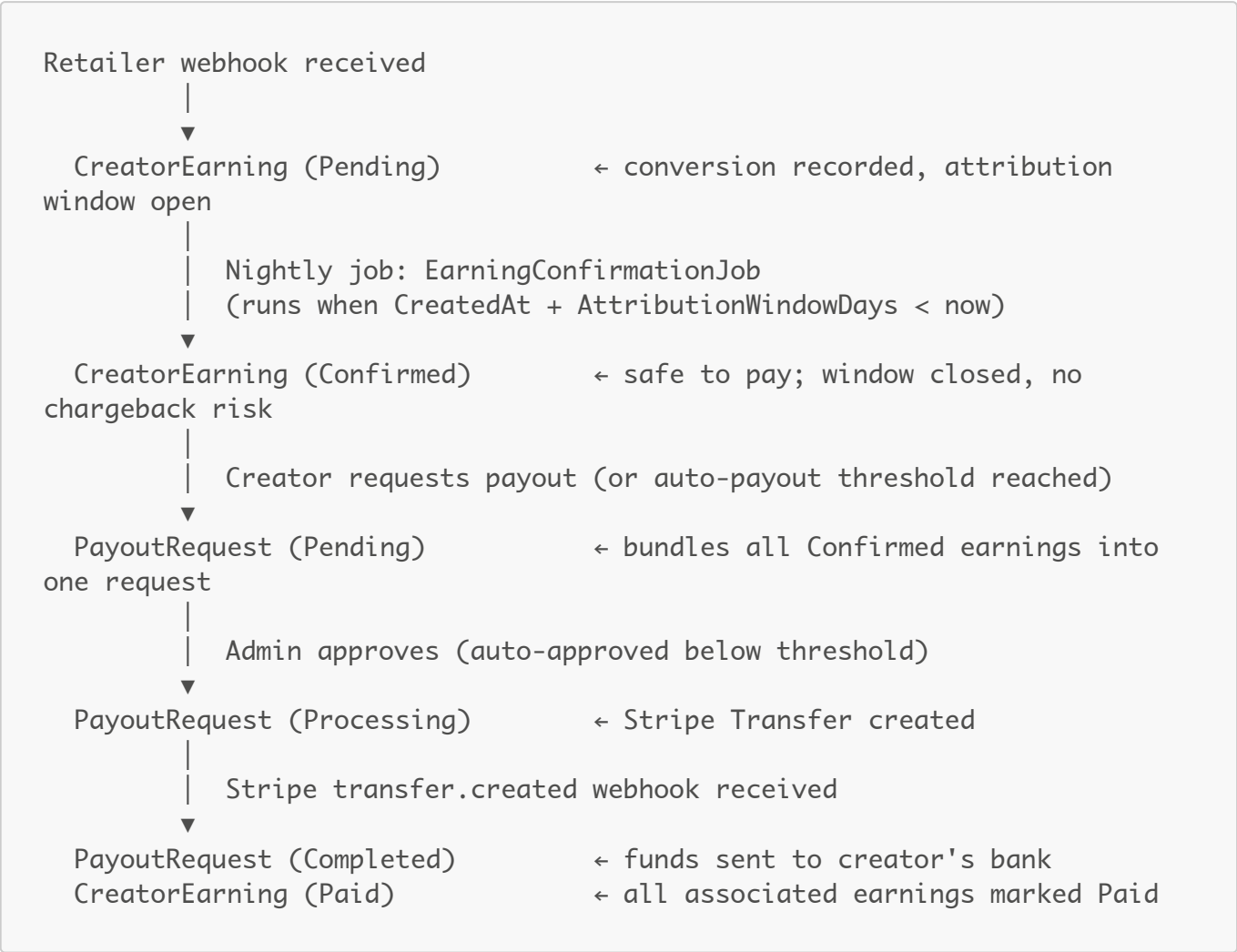
Chosen approach: Stripe Connect Express accounts.

Rationale:

- Creator KYC (identity verification, bank account collection) is fully handled by Stripe via its hosted onboarding UI — LYKE never touches bank details, reducing PCI and regulatory scope
- Express accounts support payouts to creators in 40+ countries in 135+ currencies
- Stripe's API has first-class .NET SDK support (`Stripe.net`)
- Consistent with the existing webhook pattern (`HMAC-SHA256` signature verification already in `CommerceService`)

LYKE operates as the Stripe platform account. Affiliate commissions and sponsored fees flow into LYKE's Stripe balance, and transfers are issued to each creator's connected Express account on payout.

Earning Lifecycle



Data Model Changes

New Entity: **CreatorPayoutAccount**

Stores the creator's Stripe Connect account details. One per creator.



StripeAccountId is encrypted at rest using AES-256 via a **IEncryptionService** (same pattern as existing **PayoutDetails** intent). The existing **Creator.PayoutDetails** field is superseded by this entity and should be left in place but ignored for now (migration path: null it out once onboarding is complete).

Enum: **StripeAccountStatus**

Pending	= 0	← account created, onboarding not started
Active	= 1	← KYC complete, payouts enabled
Restricted	= 2	← Stripe has restricted the account (action needed)
Suspended	= 3	← account disabled by LYKE or Stripe

New Entity: **PayoutRequest**

Tracks a single payout event that bundles multiple confirmed earnings.

```

PayoutRequest
├─ Id (GUID)
├─ CreatorId (FK → Creator)
├─ Amount (decimal)
├─ Currency (string)
├─ Status (enum)                                ← Pending | Approved | Processing
├─ Completed | Failed | Rejected
├─ StripeTransferId (string?)                  ← populated after Stripe transfer
├─ FailureReason (string?)
├─ AdminNotes (string?)
├─ RequestedAt (DateTime)
├─ ReviewedAt (DateTime?)
├─ ProcessedAt (DateTime?)
└─ Creator (navigation)
  
```

Enum: **PayoutRequestStatus**

Pending	= 0	← submitted by creator, awaiting admin review
Approved	= 1	← approved (by admin or auto-approved below threshold)
Processing	= 2	← Stripe Transfer API call made
Completed	= 3	← Stripe webhook confirmed transfer paid
Failed	= 4	← Stripe transfer failed; earnings reverted to Confirmed
Rejected	= 5	← admin rejected (e.g. fraud suspicion)

Modified Entity: **CreatorEarning**

Add a nullable FK to **PayoutRequest**:

```

CreatorEarning
├─ ... (existing fields)
└─ PayoutRequestId (Guid?, FK → PayoutRequest) ← set when earning is
    included in a payout

```

When a **PayoutRequest** enters **Failed** status, **PayoutRequestId** on all associated earnings is cleared and their **Status** is reverted to **Confirmed** so they can be included in the next payout attempt.

Modified Entity: **SponsoredPlacement**

Add a **CreatorId** FK so sponsored placements can be attributed to a specific creator:

```

SponsoredPlacement
├─ ... (existing fields)
└─ CreatorId (Guid?, FK → Creator) ← nullable; null = platform-placed,
    non-null = creator-attributed

```

New Configuration: **PayoutSettings**

```

public class PayoutSettings
{
    public const string SectionName = "Payout";

    // Auto-approve payouts below this amount; above requires admin review
    public decimal AutoApproveThreshold { get; set; } = 200.00m;

    // Number of days after conversion before earnings are confirmed
    // Should match CommerceSettings.DefaultAttributionWindowDays
    public int AttributionWindowDays { get; set; } = 30;

    // Stripe platform account secret key
    public string StripeSecretKey { get; set; } = string.Empty;

    // Stripe webhook signing secret for /api/webhooks/stripe
    public string StripeWebhookSecret { get; set; } = string.Empty;

    // Cron expression for EarningConfirmationJob (default: 2am daily)
    public string ConfirmationJobSchedule { get; set; } = "0 2 * * *";

    // Maximum number of earnings to confirm per job run (prevents timeout)
    public int ConfirmationBatchSize { get; set; } = 500;
}

```

New Service Interfaces

IPayoutService (Application layer)

```
public interface IPayoutService
{
    // Stripe Connect onboarding
    Task<string> CreateOnboardingUrlAsync(Guid userId, CancellationToken ct = default);
    Task<CreatorPayoutAccountResponse> GetPayoutAccountAsync(Guid userId,
    CancellationToken ct = default);
    Task HandleStripeAccountUpdatedAsync(string stripeAccountId,
    CancellationToken ct = default);

    // Payout requests
    Task<PayoutRequestResponse> RequestPayoutAsync(Guid userId,
    CancellationToken ct = default);
    Task<(IReadOnlyList<PayoutRequestResponse> Requests, PaginationMeta
    Meta)> GetPayoutHistoryAsync(Guid userId, PayoutHistoryRequest request,
    CancellationToken ct = default);

    // Admin actions
    Task<(IReadOnlyList<AdminPayoutResponse> Requests, PaginationMeta
    Meta)> ListPayoutRequestsAsync(AdminPayoutListRequest request,
    CancellationToken ct = default);
    Task ApprovePayoutAsync(Guid payoutRequestId, string? adminNotes,
    CancellationToken ct = default);
    Task RejectPayoutAsync(Guid payoutRequestId, string reason,
    CancellationToken ct = default);

    // Stripe webhook
    Task HandleTransferCreatedAsync(string stripeTransferId,
    CancellationToken ct = default);
    Task HandleTransferPaidAsync(string stripeTransferId, CancellationToken
    ct = default);
    Task HandleTransferFailedAsync(string stripeTransferId, string
    failureReason, CancellationToken ct = default);
}
```

IEarningConfirmationService (Application layer)

```
public interface IEarningConfirmationService
{
    // Promote Pending → Confirmed for earnings past the attribution
    window.
    // Returns count of earnings confirmed.
    Task<int> ConfirmEligibleEarningsAsync(CancellationToken ct = default);
}
```

IEncryptionService (Application layer)

```
public interface IEncryptionService
{
    string Encrypt(string plaintext);
    string Decrypt(string ciphertext);
}
```

AES-256-GCM implementation in Infrastructure, key sourced from [KeyVault](#) (already provisioned in Terraform).

New API Endpoints

Creator Endpoints

```
POST    /api/creators/v1/payouts/onboarding
  → Creates Stripe Connect Express account (if not exists), returns {
onboardingUrl }
  → Creator must be Verified before calling this

GET      /api/creators/v1/payouts/account
  → Returns CreatorPayoutAccountResponse { accountStatus,
onboardingComplete, payoutCurrency }

POST     /api/creators/v1/payouts/request
  → Bundles all Confirmed earnings into a PayoutRequest
  → Validates: account Active, confirmed balance >= MinPayoutThreshold
  → Auto-approves if amount <= AutoApproveThreshold (triggers Stripe
immediately)
  → Returns PayoutRequestResponse

GET      /api/creators/v1/payouts/history
  → Paginated list of PayoutRequest records for the creator
```

Admin Endpoints

```
GET      /api/admin/v1/payouts
  → List all PayoutRequests with filter: status, creatorId, dateRange
  → Sorted by RequestedAt desc

POST     /api/admin/v1/payouts/{id}/approve
  → Sets status Approved → Processing, initiates Stripe Transfer
  → Body: { adminNotes? }

POST     /api/admin/v1/payouts/{id}/reject
  → Sets status Rejected, earnings remain Confirmed for future payout
  → Body: { reason }
```

Webhook Endpoint

```
POST    /api/webhooks/stripe
  → Validates Stripe-Signature header (ConstructEvent with webhook secret)
  → Handles: account.updated, transfer.created, transfer.paid,
transfer.failed
  → Returns 200 immediately (idempotent processing)
```

Background Jobs: Azure Functions Timer Triggers

Consistent with the existing [Lyke.Functions](#) isolated-worker project.

EarningConfirmationFunction

- **Schedule:** `0 2 * * *` (2:00am UTC daily, configurable)
- **Logic:** Queries `CreatorEarning` where `Status = Pending` and `CreatedAt < (now - AttributionWindowDays)`. Promotes them to `Confirmed` in batches of `ConfirmationBatchSize`. Logs count confirmed.
- **No external calls** — pure DB update. Safe to retry.

AutoPayoutFunction (optional, Phase 2)

- **Schedule:** `0 6 1 * *` (6:00am UTC on the 1st of each month)
- **Logic:** For each creator with `confirmed balance >= MinPayoutThreshold` and an Active payout account, automatically submits a `PayoutRequest` and approves it if below `AutoApproveThreshold`.

Stripe Integration Details

Stripe Connect Express Onboarding Flow

1. Creator taps "Set up payouts" in app
2. POST `/api/creators/v1/payouts/onboarding`
 - PayoutService creates Stripe account via `StripeClient.Accounts.CreateAsync`
 - Creates `CreatorPayoutAccount` with `StripeAccountId` (encrypted), `Status=Pending`
 - Calls `StripeClient.AccountLinks.CreateAsync` (`type=account_onboarding`)
 - Returns `{ onboardingUrl }`
3. App opens `onboardingUrl` in in-app browser (Capacitor Browser plugin)
4. Creator completes Stripe's KYC flow (name, DOB, address, bank account)
5. Stripe redirects to app deep link: `lyke://payouts/onboarding/complete`
6. App calls GET `/api/creators/v1/payouts/account` → shows current status
7. Stripe sends `account.updated` webhook when KYC complete
 - PayoutService sets `AccountStatus = Active`, `OnboardingComplete = true`

Stripe Transfer on Payout Approval

1. Admin approves PayoutRequest (or auto-approve triggers)
2. PayoutService calls StripeClient.Transfers.CreateAsync:


```
{
    amount: <pence/cents>,
    currency: "gbp",
    destination: <StripeAccountId>,
    transfer_group: <PayoutRequest.Id>,
    metadata: { payoutRequestId, creatorId }
}
```
3. PayoutRequest.StripeTransferId = transfer.Id, Status = Processing
4. Stripe sends transfer.paid webhook
5. PayoutService sets PayoutRequest.Status = Completed, ProcessedAt = now
6. All associated CreatorEarning records: Status = Paid, PaidAt = now
7. Email notification sent to creator: "Your payout of £X.XX has been sent"

Stripe Transfer Failure

1. Stripe sends transfer.failed webhook
2. PayoutService sets PayoutRequest.Status = Failed, FailureReason = <stripe message>
3. Associated CreatorEarning.PayoutRequestId = null, Status reverted to Confirmed
4. Email notification sent to creator: "Your payout failed – please check your account"
5. Creator can retry by calling POST /api/creators/v1/payouts/request again

Sponsored Earnings

The **SponsoredPlacement** entity currently tracks retailer ad campaigns but does not generate **CreatorEarning** records. The connection requires:

1. **SponsoredPlacement.CreatorId FK** — links a campaign to a specific creator who will be compensated
2. **Campaign serve event** — when a sponsored post impression is served (tracked via **EngagementType.View** on a post linked to an active placement), create a **CreatorEarning** of **EarningType.Sponsored**
3. **Budget deduction** — deduct **SpentAmount** from **SponsoredPlacement.BudgetAmount**; deactivate placement when budget exhausted
4. **Fixed fee vs CPM** — two models:
 - **Fixed fee**: agreed upfront, single **CreatorEarning** created when campaign starts (Status=Confirmed immediately, no attribution window needed)
 - **CPM (cost per mille)**: earning created per 1000 impressions served, goes through normal Pending→Confirmed flow

For MVP, implement **fixed fee only** as it is simpler and avoids impression fraud concerns.

Security Considerations

Concern	Mitigation
<code>StripeAccountId</code> exposure	Encrypted at rest (AES-256-GCM), decrypted only in <code>IPayoutService</code> impl
Stripe webhook spoofing	<code>StripeClient.ConstructEvent</code> validates <code>Stripe-Signature</code> HMAC
Double-payout	<code>PayoutRequest</code> creation checks for existing <code>Pending/Processing</code> request; <code>CreatorEarning.PayoutRequestId</code> prevents double-inclusion
Payout to unverified creator	<code>POST /payouts/onboarding</code> requires <code>VerificationStatus == Approved</code>
Payout account not Active	<code>POST /payouts/request</code> validates <code>AccountStatus == Active</code>
Earnings manipulation	<code>CreatorEarning</code> records are system-generated only (no creator write endpoint)
GDPR	Stripe stores PII/bank info under its own DPA; <code>StripeAccountId</code> stored encrypted; on GDPR deletion, call <code>Stripe.Accounts.DeleteAsync</code> and null out <code>StripeAccountId</code>

Email Notifications (new templates)

Event	Template name	Recipient
Payout request received	<code>payout-requested.liquid</code>	Creator
Payout approved (above threshold)	<code>payout-approved.liquid</code>	Creator
Payout completed	<code>payout-completed.liquid</code>	Creator
Payout failed	<code>payout-failed.liquid</code>	Creator
Payout rejected by admin	<code>payout-rejected.liquid</code>	Creator
Payout requires admin review	<code>payout-admin-review.liquid</code>	Admin

Frontend Changes

Creator — Earnings Page (update existing)

- Add "Set up payouts" CTA when `onboardingComplete == false`
- Show payout account status badge (Pending / Active / Restricted)
- "Request payout" button: enabled when `eligibleForPayout && accountStatus == Active`
- Payout history tab: list of `PayoutRequest` records with status and amount

Creator — Payout Onboarding Page (new)

- Explains what Stripe Connect is and what data Stripe will collect
- "Connect bank account" button → opens `onboardingUrl` in in-app browser
- Returns to app via deep link `lyke://payouts/onboarding/complete`
- Polls `GET /api/creators/v1/payouts/account` every 3s for up to 30s to confirm `onboardingComplete`

Admin — Payout Management Page (new)

- Tab-based: Pending Review / All
 - Each row: creator name, amount, requested date, status badge, Approve/Reject actions
 - Approve modal: optional admin notes field
 - Reject modal: required reason field
-

Open Questions / Out of Scope for MVP

1. **Tax reporting (1099/HMRC)** — Stripe provides tax forms for US creators via the dashboard. UK creators will need annual summaries. Out of scope for now.
2. **Multi-currency conversion** — Stripe handles FX for international creators. LYKE always initiates transfers in the creator's `PayoutCurrency`. Rate risk is Stripe's.
3. **Chargeback handling** — if a retailer reverses a commission after an earnings record is already `Confirmed` or `Paid`, the current design has no debt recovery mechanism. Mitigation: extend attribution window, add a `Clawback` earning type in future.
4. **Auto-payout (scheduled)** — `AutoPayoutFunction` described above is Phase 2; MVP requires creator to manually request payout.
5. **Minimum payout frequency cap** — prevent creators requesting payout more than once per 7 days (spam mitigation). Add to `POST /payouts/request` validation in Phase 2.